### Ejercicio 1: Tipo Autor (0,5 ptos)

```
public record Autor (String nombre, String afiliacion, Boolean correspondencia) {
	public Autor {
		Checkers.check("El nombre del autor (" +nombre +") no puede empezar por *",
				!nombre.startsWith("*"));
	}

	public int hashCode() {
		return Objects.hash(afiliacion, nombre);
	}

	//Dos autores son iguales si tienen el mismo nombre y la misma afiliación
	public boolean equals(Object obj) {
		if (this == obj)
			return true;
		if (obj == null)
			return false;
		if (getClass() != obj.getClass())
			return false;
		Autor other = (Autor) obj;
		return Objects.equals(afiliacion, other.afiliacion) &&
			Objects.equals(nombre, other.nombre);
	}

}
```

### Ejercicio 2: Tipo Articulo (2 ptos)

```
public class Articulo implements Comparable<Articulo> {

	private String titulo;
	private List<Autor> autores;
	private LocalDate fechaPublicacion;
	private String revista;
	private TipoPublicacion tipo;
	private Integer numCitas;
	private SortedSet<String> keywords;


	public Articulo(String titulo, List<Autor> autores,  LocalDate fechaPublicacion,
			String revista, TipoPublicacion tipo) {
		this.titulo = titulo;
		String msg = String.format(
			"Debe haber  haber un solo autor de correspondencia <%s>", titulo);
		Checkers.check(msg, haySoloUnoCorrespondencia(autores));
		this.autores = autores;
		setFechaPublicacion(fechaPublicacion);
		this.revista = revista;
		this.tipo = tipo;
		this.numCitas = 0;
		this.keywords= new TreeSet<>();
	}


	private Boolean haySoloUnoCorrespondencia(List<Autor> autores) {
		Integer cont = 0;
		for (Autor a: autores) {
			if (a.correspondencia()) {
				cont++;
```

```
                }
            }
            return cont.equals(1);
    }

    public void setNumCitas(Integer numCitas) {
            Checkers.check("numCitas >= 0", numCitas >= 0);
            this.numCitas = numCitas;
    }

    public List<Autor> getAutores() {
            return new ArrayList<Autor>(autores);
    }
    public Integer getNumCitas() {
            return numCitas;
    }

    public String getTitulo() {
            return titulo;
    }

    public LocalDate getFechaPublicacion() {
            return fechaPublicacion;
    }

    public void setFechaPublicacion(LocalDate fechaPublicacion) {
            // OJO: fechaPublicacion.isBefore(LocalDate.now()) NO cumple el enunciado
            // porque no tiene en cuenta el día de hoy
            Checkers.check ("Fecha publicación incorrecta",
                            LocalDate.now().isAfter(fechaPublicacion));
            this.fechaPublicacion = fechaPublicacion;
    }

    public String getRevista() {
            return revista;
    }

    public TipoPublicacion getTipo() {
            return tipo;
    }

    public SortedSet<String> getKeywords() {
            return new TreeSet<String>(keywords);
    }

    public void agregaKeyword(String keyword) {
            keywords.add(keyword.toLowerCase());
    }

    public String getPrimerAutor() {
            return autores.get(0).nombre();
    }

    public Integer getNumeroAutores() {
            return autores.size();
    }


    public Boolean esAutor(String nombre) {
            Boolean res = false;
```

```
		for(Autor a: autores) {
			if (a.nombre().equals(nombre)) {
				res = true;
				break;
			}
		}
		return res;
	}

	public Autor autorCorrespondencia() {
		Autor res = null;
		for (Autor a: autores) {
			if (a.correspondencia()) {
				res = a;
				break;
			}
		}
		return res;

	}

	public int compareTo(Articulo a) {
		int res = getFechaPublicacion().compareTo(a.getFechaPublicacion());
		if (res == 0) {
			res = getTitulo().compareTo(a.getTitulo());
			if (res == 0) {
				res =getRevista().compareTo(a.getRevista());
			}
		}
		return res;
	}


	@Override
	public String toString() {
		return  titulo + ", " + autores + ", " + fechaPublicacion
				+ ", " + revista + ", " + tipo + ", " + numCitas + ", " +
				keywords ;
	}

	@Override
	public int hashCode() {
		return Objects.hash(revista, fechaPublicacion, titulo);
	}

	@Override
	public boolean equals(Object obj) {
		if (this == obj)
			return true;
		if (obj == null)
			return false;
		if (getClass() != obj.getClass())
			return false;
		Articulo other = (Articulo) obj;
		return Objects.equals(revista, other.revista) &&
				Objects.equals(fechaPublicacion, other.fechaPublicacion)
				&& Objects.equals(titulo, other.titulo);
	}
}
```

### Ejercicio 3: Tipo ArticuloPago (0,5 ptos)

```
public class ArticuloPago extends Articulo {
	private Double precio;

	public ArticuloPago(String titulo, List<Autor> autores,
			LocalDate fechaPublicacion, String revista, Double precio) {
		super(titulo, autores, fechaPublicacion, revista, TipoPublicacion.PAGO);
		setPrecio(precio);
	}

	public Double getPrecio() {
		return precio;
	}

	public void setPrecio(Double precio) {
		Checkers.check("El precio no puede ser negativo", precio>=0);
		this.precio = precio;
	}

	public String toString() {
		return super.toString()+"," + precio ;
	}

}
```

### Ejercicio 4: Factoría (1,5 ptos)

```
private static final String SEPARADOR_PPAL = ",";
private static final String SEPARADOR_SECUND = ";";
private static final String SEPARADOR_AUTOR = "-";

public static Articulo parseaArticulo(String linea) {
	Articulo res = null;
	String [] trozos = linea.split(SEPARADOR_PPAL);
	String msg = String.format("Formato no válido (%d) <%s>", trozos.length, linea);
	Checkers.check(msg, trozos.length == 7 || trozos.length==8);

	String titulo = trozos[0].strip();
	List<Autor> autores = parseaAutores(trozos[1].strip());
	LocalDate fechaPublicacion = LocalDate.parse(trozos[2].strip());
	String revista = trozos[3].strip();
	TipoPublicacion tipo = TipoPublicacion.valueOf(trozos[4].strip().toUpperCase());
	Integer numCitas = Integer.parseInt(trozos[5].strip());
	SortedSet<String> keywords = parseaKeywords(trozos[6].strip());
	if (trozos.length == 7) {
		res = new Articulo(titulo, autores, fechaPublicacion, revista, tipo);
	} else if (trozos.length == 8){
		Double precio = Double.parseDouble(trozos[7].strip());
		res = new ArticuloPago(titulo, autores, fechaPublicacion, revista, precio);
	}
	res.setNumCitas(numCitas);
	for (String keyword: keywords) {
		res.agregaKeyword(keyword);
	}
	return res;
```

```
private static List<Autor> parseaAutores(String autoresStr) {
	List<Autor> autores = new ArrayList<>();
	String [] trozos = autoresStr.split(SEPARADOR_SECUND);
	for (String trozo: trozos) {
		autores.add(parseaAutor(trozo));
	}
	return autores;
}

private static Autor parseaAutor(String autorStr) {
	String [] trozos = autorStr.split(SEPARADOR_AUTOR);
	String msg = String.format("Formato Autor no válido (%d) <%s>", trozos.length,
					autorStr);
	Checkers.check(msg, trozos.length==2);
	String nombre = trozos[0].trim();
	String afiliacion = trozos[1].trim();
	Boolean corresponding = false;
	if (nombre.startsWith("*")) {
		corresponding = true;
		nombre=nombre.replace("*", "");
	}
	return new Autor(nombre, afiliacion, corresponding);
}

private static SortedSet<String> parseaKeywords(String keywordsStr) {
	String [] trozos= keywordsStr.split(";");
		SortedSet<String> ss = new TreeSet<>();
		for (String trozo:trozos) {
			ss.add(trozo.strip());
		}
		return ss;
	}

}
```

**Ejercicio 5: Tipo contenedor PubIndex (5,5 ptos)**

1. *porcentajePrimerAutor:* dado un autor, el porcentaje de artículos en los que dicho autor es primer autor, calculado sobre el total de sus participaciones como autor. (0,75 ptos)

```
public Double porcentajePrimerAutor(String autor) {
    Long articulosPrimerAutor = articulos.stream()
                                    .filter(art->art.getPrimerAutor().equals(autor))
                                    .count();
    Long totalArticulos = articulos.stream()
                        .filter(art->art.esAutor(autor))
                        .count();
    Double res = 0.0;
    if (totalArticulos > 0) {
        res= articulosPrimerAutor*100. / totalArticulos;
    }
    return res;
}
```

2. *autorMasCitado*: dada una keyword como parámetro, devuelve el nombre del autor que acumula el mayor número total de citas considerando exclusivamente los artículos que contienen esa keyword..

Si no se puede calcular el más citado, devuelve null. **Implementar sin usar Streams ni Collections** (1 pto)

```
public String autorMasCitado(String keyword){
          //1. Calcular el total de citas por autor (map acumulador)
          Map<Autor, Integer> m = new HashMap<>();
          for (Articulo a: articulos) {
                if (a.getKeywords().contains(keyword)) {
                      for (Autor autor: a.getAutores()) {
                            Autor clave = autor;
                            Integer valor = m.getOrDefault(clave, 0);
                            m.put(clave, valor + a.getNumCitas());
                      }
                }
          }

          //Cálculo del máximo
          Map.Entry<Autor, Integer> masCitado = null;
          for (Map.Entry<Autor, Integer> e: m.entrySet()) {
                if (masCitado == null || masCitado.getValue() < e.getValue()) {
                      masCitado = e;
                }
          }
          String res = null;
          if (masCitado != null) {
                Autor a = masCitado.getKey();
                res = a.nombre() + " (" + a.afiliacion()+")";
          }
          return res;
}
```

3. *revistasOrdenadasPorImpactoMedio*: devuelve una lista de nombres de revistas ordenadas por el promedio de citas (de menor a mayor) de los artículos publicados en esa revista. Si hay dos revistas con el mismo promedio de citas, aparecerán en orden alfabético. (1,25 ptos)

```
public List<String> revistasOrdenadasPorImpactoMedio() {
     Map<String, Double> m = articulos.stream()
          .collect(Collectors.groupingBy(
              Articulo::getRevista,
              Collectors.averagingInt(Articulo::getNumCitas)
         ));

    Comparator<Map.Entry<String, Double>> cv = Map.Entry.comparingByValue();
    Comparator<Map.Entry<String, Double>> c =
                  cv.thenComparing(Map.Entry.comparingByKey());
    return m.entrySet()
           .stream()
           .sorted(c)
           .map(Map.Entry::getKey)
           .collect(Collectors.toList());
}
```

4. *titulosPrecioSuperiorMedio*: devuelve un SortedSet que contiene los títulos de los artículos de pago cuyo precio está por encima de la media de todos los artículos de pago. (1,25 ptos)

```
public SortedSet<String> titulosPrecioSuperiorMedio() {
```

```
    Double precioMedio = articulos.stream()
            .filter(a -> a instanceof ArticuloPago)
            .map(a -> (ArticuloPago) a)
            .mapToDouble(ArticuloPago::getPrecio)
            .average()
            .orElse(0);

    return articulos.stream()
            .filter(a -> a instanceof ArticuloPago)
            .map(a -> (ArticuloPago) a)
            .filter(a->a.getPrecio()>precioMedio)
            .map(ArticuloPago::getTitulo)
            .collect(Collectors.toCollection(TreeSet::new));
    }
```

5. *autorMasProductivo*: dado un año, devuelve el Autor que ha escrito más artículos dicho año. Si el año es null, se consideran todos los años disponibles. Si no se puede calcular, devuelve *null* (1,25 ptos)

```
public Autor autorMasProductivo (Integer anyo) {
    Predicate<Articulo> pa = a-> anyo == null
                               || a.getFechaPublicacion().getYear() == anyo;
    Map<Autor, Long>m= articulos.stream()
                            .filter(pa )
                            .flatMap(art -> art.getAutores().stream())
                            .collect(Collectors.groupingBy(
                                        Function.identity(),
                                       Collectors.counting()
                        ));

     return m.entrySet().stream()
                        .max(Map.Entry.comparingByValue())
                        .map(Map.Entry::getKey)
                        .orElse(null);
}
```